Эмуляция и измерение потерь пакетов в глобальных сетях

Воспроизводимый эксперимент NETEM

Выполнил: Базлов В.А.

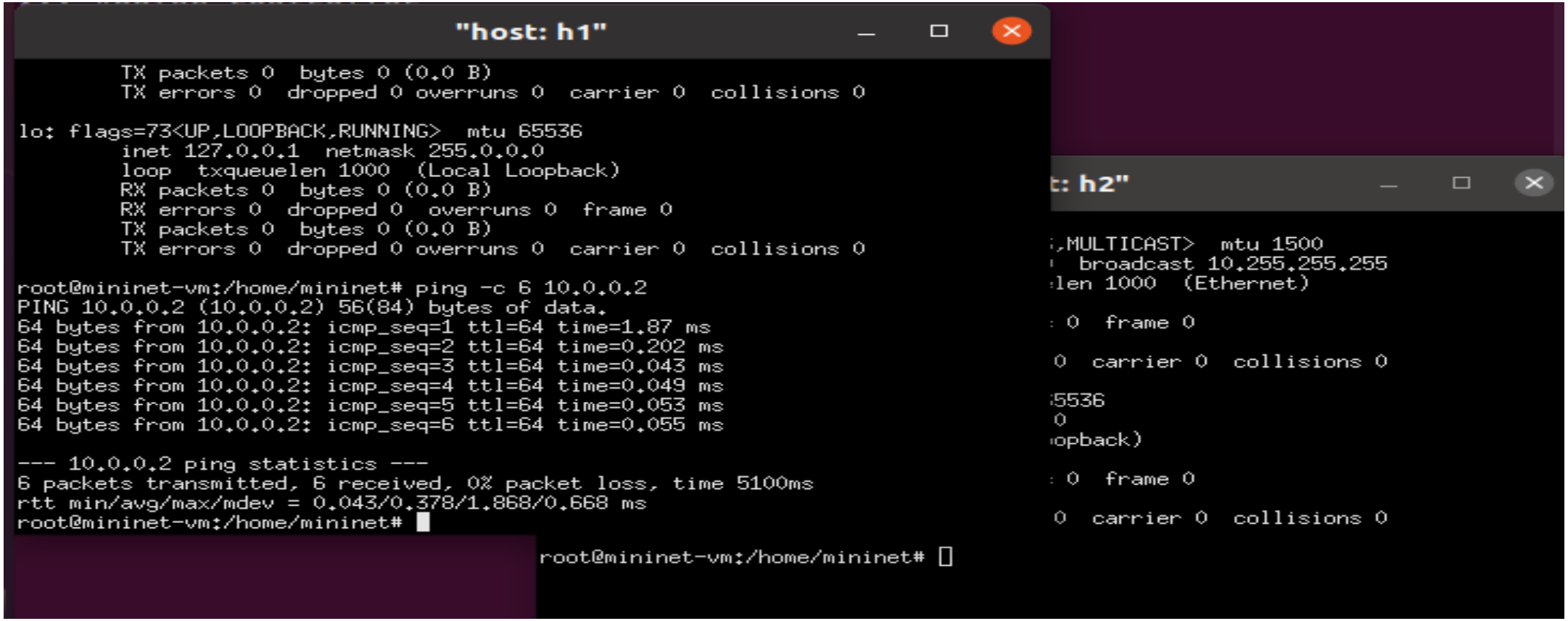
Отчёт по лабораторной работе №5

Цель и задания

Я исследовал влияние потери, дублирования, переупорядочивания и повреждения пакетов в сети Mininet с помощью механизма NETEM.

h1 — 10.0.0.1

h2 — 10.0.0.2



Я проверил доступность хоста h2 с хоста h1 и убедился в исходной связности сети.

```
64 bytes from 10.0.0.2: icmp_seq=91 ttl=64 time=0.061 ms
64 bytes from 10.0.0.2: icmp_seq=92 ttl=64 time=0.055 ms
64 bytes from 10.0.0.2: icmp_seq=93 ttl=64 time=0.072 ms
64 bytes from 10.0.0.2: icmp_seq=94 ttl=64 time=0.047 ms
64 bytes from 10.0.0.2: icmp_seq=95 ttl=64 time=0.056 ms
64 bytes from 10.0.0.2: icmp_seq=96 ttl=64 time=0.056 ms
64 bytes from 10.0.0.2: icmp_seq=97 ttl=64 time=0.057 ms
64 bytes from 10.0.0.2: icmp_seq=99 ttl=64 time=0.052 ms
64 bytes from 10.0.0.2: icmp_seq=100 ttl=64 time=0.274 ms

--- 10.0.0.2 ping statistics ---
100 packets transmitted, 86 received, 14% packet loss, time 101326ms
rtt min/avg/max/mdev = 0.039/0.083/0.710/0.107 ms
root@mininet-vm:/home/mininet#
```

Я настроил потерю 10% пакетов на исходящем интерфейсе h1 и проверил передачу 100 ICMP-пакетов.

Потеря

```
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=0.052 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=0.051 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=0.070 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=0.059 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=0.052 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=0.050 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=0.051 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19453ms
rtt min/avg/max/mdev = 0.034/0.104/0.671/0.152 ms
root@mininet-vm:/home/mininet#
```

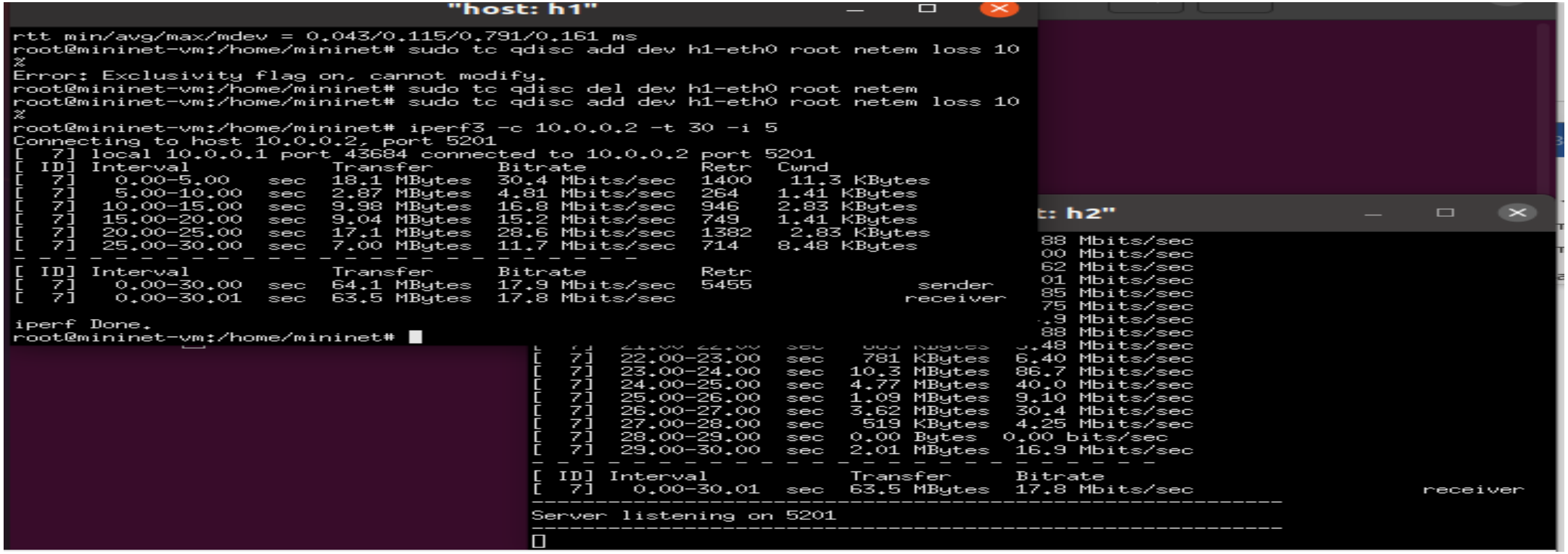
Я добавил потерю 10% пакетов также на интерфейсе h2 и повторно проверил связь между хостами.

Потери

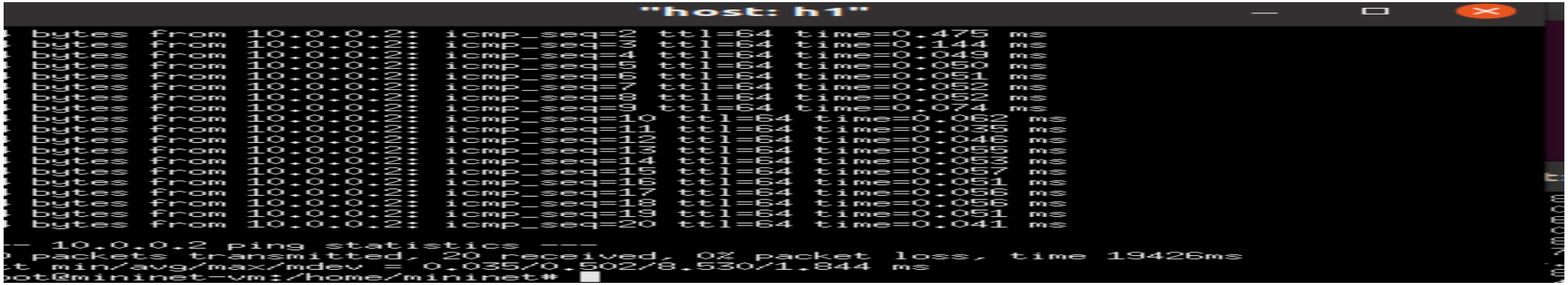
```
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=0.059 ms
64 bytes from 10.0.0.2: icmp_seq=23 ttl=64 time=0.053 ms
64 bytes from 10.0.0.2: icmp_seq=24 ttl=64 time=0.074 ms
64 bytes from 10.0.0.2: icmp_seq=26 ttl=64 time=0.045 ms
64 bytes from 10.0.0.2: icmp_seq=27 ttl=64 time=0.051 ms
64 bytes from 10.0.0.2: icmp_seq=28 ttl=64 time=0.054 ms
64 bytes from 10.0.0.2: icmp_seq=29 ttl=64 time=0.053 ms
64 bytes from 10.0.0.2: icmp_seq=30 ttl=64 time=0.087 ms
64 bytes from 10.0.0.2: icmp_seq=33 ttl=64 time=0.061 ms
64 bytes from 10.0.0.2: icmp_seq=34 ttl=64 time=0.558 ms
64 bytes from 10.0.0.2: icmp_seq=35 ttl=64 time=0.074 ms
64 bytes from 10.0.0.2: icmp_seq=36 ttl=64 time=0.066 ms
64 bytes from 10.0.0.2: icmp_seq=37 ttl=64 time=0.055 ms
64 bytes from 10.0.0.2: icmp_seq=38 ttl=64 time=0.052 ms
64 bytes from 10.0.0.2: icmp_seq=42 ttl=64 time=0.051 ms
64 bytes from 10.0.0.2: icmp_seq=44 ttl=64 time=0.051 ms
64 bytes from 10.0.0.2: icmp_seq=45 ttl=64 time=0.051 ms
64 bytes from 10.0.0.2: icmp_seq=46 ttl=64 time=0.244 ms
64 bytes from 10.0.0.2: icmp_seq=48 ttl=64 time=0.057 ms

--- 10.0.0.2 ping statistics ---
50 packets transmitted, 32 received, 36% packet loss, time 50139ms
rtt min/avg/max/mdev = 0.043/0.115/0.791/0.161 ms
root@mininet-vm:/home/mininet#
```

Я настроил вероятность потери 50% с корреляцией 50%. Затем отправил 50 ICMP-пакетов и проанализировал пропуски в последовательности icmp_seq.



Я настроил повреждение пакетов на интерфейсе h1. Для проверки TCP-трафика запустил сервер и клиент iPerf3 и изучил число повторных передач Retr.



Я задал задержку 10 мс и переупорядочивание 25% с корреляцией 50%. По ответам ping сравнил наблюдаемые значения времени доставки.

```
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.300 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.301 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.049 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.042 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.053 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.054 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=0.060 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=0.061 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=0.057 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=0.059 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=0.059 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=0.060 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=0.055 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=0.055 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=0.054 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=0.050 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=0.051 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=0.067 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, +8 duplicates, 0% packet loss, time 19393ms
rtt min/avg/max/mdev = 0.035/0.429/9.074/1.669 ms
root@mininet-vm:/home/mininet#
```

Я настроил дублирование 50% пакетов. В выводе ping обнаруживал повторные ответы, помеченные как DUP!, и оценивал их количество.

```
mininet@mininet-vm:~$ cd ~/work/lab_netem_ii/simple-drop
mininet@mininet-vm:~/work/lab_netem_ii/simple-drop$ nano lab_netem_ii.py
mininet@mininet-vm:~/work/lab_netem_ii/simple-drop$ nano Makefile
mininet@mininet-vm:~/work/lab_netem_ii/simple-drop$ make
sudo python lab_netem_ii.py
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s
*** Waiting for switches to connect
s1
*** Set loss
*** h1 : ('tc qdisc add dev h1-eth0 root netem loss 10%',)
*** h2 : ('tc qdisc add dev h2-eth0 root netem loss 10%',)
*** Ping
```

Terminal

Я подготовил Python-скрипт на основе Mininet API. Он автоматически создавал топологию, применял выбранное правило NETEM, выполнял проверку и сохранял результаты.

```
mininet@mininet-vm:~/work/lab_netem_ii/correlation-drop$ cp ../simple-drop/Make
file .
mininet@mininet-vm:~/work/lab_netem_ii/correlation-drop$ nano lab_netem_ii.py
mininet@mininet-vm:~/work/lab_netem_ii/correlation-drop$ make
sudo python lab_netem_ii.py
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Set loss
*** h1 : ('tc qdisc add dev h1-eth0 root netem loss 50%',)
*** h2 : ('tc qdisc add dev h2-eth0 root netem loss 50%',)
*** Ping
```

Я подготовил Makefile для запуска разных режимов эксперимента и очистки файлов с результатами.

```
mininet@mininet-vm:~/work/lab_netem_ii/corrupt$ cp ../simple-drop/Makef
mininet@mininet-vm:~/work/lab_netem_ii/corrupt$ nano lab_netem_ii.py
mininet@mininet-vm:~/work/lab_netem_ii/corrupt$ make
sudo python lab_netem_ii.py
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Set loss
*** h2 : ('tc qdisc add dev h2-eth0 root netem loss 10%',)
*** Ping
```

Запуск э

Для режимов на основе ping скрипт сохранял вывод в ping.txt и выводил число отправленных и полученных пакетов, а также процент потерь. Для режима corrupt сохранялся JSON-отчёт iPerf3.

```
mininet@mininet-vm:~/work/lab_netem_ii/corrupt$ cat Makefile
all: ping_full.log

ping_full.log: lab_netem_ii.py
	sudo python lab_netem_ii.py
	sudo chown mininet:mininet ping_full.log

clean:
	-rm -f *.log
mininet@mininet-vm:~/work/lab_netem_ii/corrupt$ make
sudo python lab_netem_ii.py
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Set loss
*** h2 : ('tc qdisc add dev h2-eth0 root netem loss 10%',)
*** Ping
```

Вывод

В ходе лабораторной работы я создал тестовую сеть Mininet из двух хостов и коммутатора. С помощью NETEM исследовал потерю, корреляцию потерь, повреждение, переупорядочивание и дублирование пакетов. Проверку выполнял средствами ping и iPerf3. Также я подготовил воспроизводимый Python-эксперимент на основе Mininet API и Makefile для его запуска.